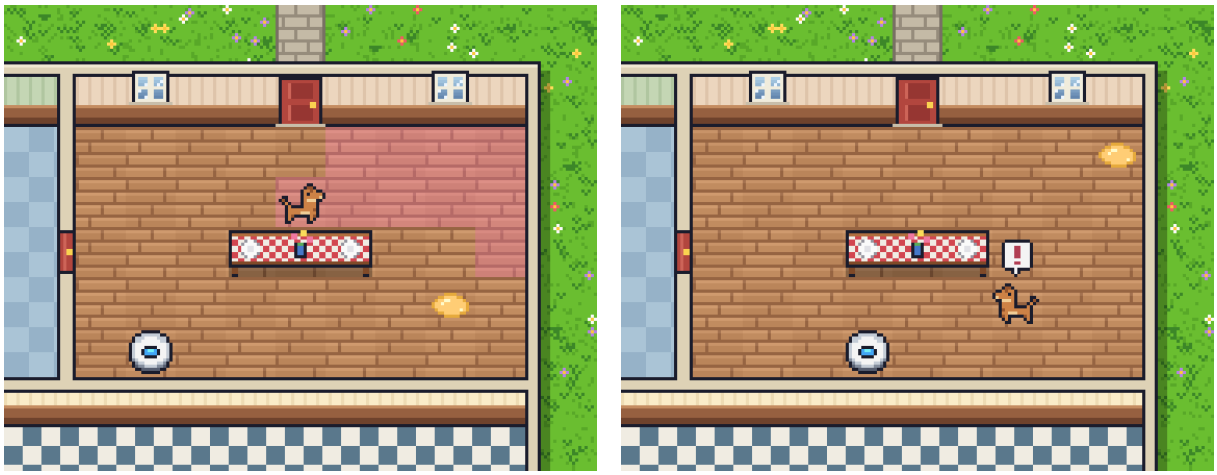
Exercício 1: Tópicos Especiais em Aprendizado por Reforço

COS767 - 2026/3

1 Cenário

Vamos programar um robô de limpeza que divide o espaço com um cachorro territorialista! O cachorro se movimenta pela sala "marcando território" e cabe ao robô limpar a bagunça. O robô é penalizado sempre que a sala está suja, então ele deve passar onde o cachorro molhou o mais rápido possível! Para dificultar a história, o cachorro também é brincalhão, e se percebe o robô se movendo, ele o persegue, e pula em cima dele. Isso faz com que o robô precise reiniciar, ficando k rodadas parado enquanto o cachorro anda livremente pela sala.



(a) (b)

Figura 1: A sala, contendo uma mesa no centro. (a) Cachorro em modo normal, olhando para a direita: as células em rosa indicam o seu campo de visão. (b) Cachorro em modo brincadeira ("!"), perseguindo o robô.

2 Descrição do MDP

Sala. Uma grade de 9×5 células com uma mesa de 3×1 no centro, que bloqueia o movimento e a visão. Ou seja, há 42 células livres para o robô e para o cachorro.

Estado. Posição do robô, posição do cachorro, orientação do cachorro (N, L, S ou O), modo do cachorro (normal ou brincadeira) e posição do xixi (ou sala limpa). O cachorro, sendo territorialista, só faz xixi novamente quando nota que a sala está limpa, ou seja, há no máximo um xixi na sala por vez. Obtemos então $2 \cdot 42 \cdot 42 \cdot 4 \cdot 43 = 606.816$ estados.

Ações. Movimentar o robô uma célula para N, L, S ou O, ou ficar parado. Um movimento contra a parede ou a mesa não surte efeito.

Recompensa. $r(s) = -1$ sempre que há xixi na sala e $r(s) = 0$ caso contrário.

Uma rodada. A partir do estado s e da ação a, os eventos acontecem na seguinte ordem:

1. O robô executa a, uma ação de movimento. Para limpar o xixi, basta ele terminar a rodada na mesma célula.

2. Se o cachorro está em modo normal, ele percebe o robô quando o robô se moveu nesta rodada e (i) a nova posição do robô está no campo de visão do cachorro, ou (ii) o robô entrou na célula do cachorro. O campo de visão é um cone de 90° na direção em que o cachorro olha, limitado pelo obstáculo central, a mesa. Ao perceber o robô, o cachorro entra no modo brincadeira já nesta rodada.

3. O cachorro se move.

   - **Modo normal.** Com probabilidade $1-p$, anda uma célula na direção que está virado. Se está de frente para uma barreira, o cachorro dá meia-volta. Com probabilidade p, o cachorro para e vira para uma das outras três direções, escolhida uniformemente, e, se a sala está limpa, faz xixi na própria célula.
   - **Modo brincadeira.** Se o cachorro tem linha de visão até o robô (em qualquer direção), avança até duas células em direção a ele, incluindo diagonais, e passa a olhar para ele. Caso contrário, corre até duas células na direção em que estava olhando, ou seja, na direção em que viu o robô pela última vez. Se ele chega a um obstáculo sem ver o robô, volta ao modo normal.

4. Se o cachorro alcança a célula do robô, há uma captura. O cachorro volta ao modo normal e o robô é desligado! Enquanto o robô reinicia, o cachorro anda k rodadas em modo normal (e pode fazer xixi).

As k rodadas de uma captura formam uma única transição, com recompensa $r(s)$. Dessa forma, a propriedade de Markov é mantida: cada valor de k apenas define uma distribuição para o próximo estado, e o robô não precisa conhecer o caminho que o cachorro fez.

Parâmetros. $p = 0{,}2$. Os experimentos devem usar pelo menos $k \in \{0, 1, 2, 4, 6, 10\}$ e dois fatores de desconto: $\gamma = 0{,}7$ (curto prazo) e $\gamma = 0{,}99$ (longo prazo).

3 Código fornecido

O repositório já contém o ambiente, o modelo e a animação. O projeto é em Python, e usa o gerenciador de pacotes `uv`. A instalação pode ser feita com `uv sync`. Se o(a) estudante tiver a preferência de outro gerenciador de pacotes ou linguagem de programação, é permitido o uso de ferramentas de IA para a conversão do código. Os componentes a serem implementados como parte do exercício, porém, devem ser feitos pela própria pessoa. Por isso a maior parte do código auxiliar já está disponibilizada, para que só se preocupe com os experimentos e algoritmos relevantes à disciplina.

Para ajudar a depurar as implementações, use a função `animate(policy, env, steps, filename)` para salvar um GIF da sua política em ação (badum tss)!

Módulo	Conteúdo
`cleaning_robot/env.py`	`CleaningEnv`: dinâmica do cachorro e interface do MDP
`cleaning_robot/model.py`	`all_states`, `state_index`, `TabularModel`
`cleaning_robot/policy.py`	`PolicyMixin`, `GreedyPolicy`, `RandomPolicy`
`cleaning_robot/animate.py`	`animate` (GIF de uma política) e `rollout` (simulação)
`cleaning_robot/dp.py`	`TabularPolicy`, `tabulate` e os esqueletos dos itens 1 e 2
`cleaning_robot/qlearning.py`	esqueleto do item 3

```python
from cleaning_robot.env import CleaningEnv
from cleaning_robot.model import TabularModel, state_index
from cleaning_robot.policy import GreedyPolicy
from cleaning_robot.animate import animate, rollout

env = CleaningEnv(k=2) # p = 0.2
s = env.random_init(seed=0) # estado inicial aleatorio
env.reward(s) # r(s)
env.next_state_probs(s, a) # {s': P(s'|s,a)}
env.next_state_sample(s, a, seed) # s' ~ P(.|s,a)

model = TabularModel(env) # model.P[a][i, j] = P(j|i,a), model.R[i] = r(i)
i = state_index(env, s) # linha de s em P e R

animate(GreedyPolicy(env), env, steps=80, filename="greedy.gif")
```

Modelo. Para facilitar uma implementação matricial dos algoritmos model-based, fornecemos a classe `TabularModel`, que enumera os estados com `state_index` e guarda o MDP como uma matriz esparsa $n \times n$ por ação, `model.P[a]`, e o vetor de recompensas `model.R`. Pode valer a pena salvar as matrizes em disco (`scipy.sparse.save_npz`).

Políticas. Toda política deve herdar `PolicyMixin` e implementar `act(state, seed=None)`, que devolve uma ação. `TabularPolicy` guarda uma ação por índice de estado; `tabulate(model, policy)` produz essa tabela para qualquer política determinística. A política gulosa (`GreedyPolicy`) segue o caminho mais curto até o xixi e ignora o cachorro; a política aleatória (`RandomPolicy`) escolhe uma ação uniforme.

4 Funções e classes a implementar

- `policy_evaluation(model, policy, gamma)` em `dp.py`: o valor V^π de uma política, usando o modelo. A política é uma ação por estado (determinística) ou uma matriz de probabilidades $n \times 5$.
- `ValueIteration(model, gamma)` e `PolicyIteration(model, gamma)` em `dp.py`.
- `QLearning(env, gamma, n_steps, seed)` em `qlearning.py`. Lembre que o Q-learning é model-free e só pode usar `random_init`, `next_state_sample` e `reward`, não `model.P`.

As classes já têm um esqueleto, e herdam de `PolicyMixin`, mas é permitido mexer no código já feito, inclusive nos construtores para incluir parâmetros adicionais.

5 Tarefas

Para os itens que seguem abaixo, você deve definir e justificar suas métricas de desempenho. Vamos querer medir convergência para o ótimo, comparar políticas, etc.

Os itens 1 e 2 devem ser feitos para $\gamma = 0{,}7$ e $\gamma = 0{,}99$.

1. **Avaliação de políticas.** Implemente `policy_evaluation` e avalie a política gulosa e a política aleatória para cada k.
   (a) Quanto uma política perde em relação à outra, como medir isso?
   (b) Como o valor de cada uma varia com k? E com γ?

(c) Qual abordagem é mais apropriada neste cenário: resolver o sistema diretamente ou por um método iterativo? Justifique comparando tempo e memória.

2. **Policy iteration e value iteration.** Implemente os dois métodos e faça gráficos do tempo de convergência e do número de iterações em função de k.

   (a) O tempo de convergência muda muito com k? E o número de iterações? Por quê?

   (b) A política ótima muda com k? Como medir isso?

   (c) Compare a política ótima com as políticas do item 1.

3. **Q-learning.** Escolha os dois cenários (k, γ) mais interessantes e execute o Q-learning.

   (a) Como inicializar Q? Considere que só existem recompensas não-positivas. Isso tem alguma consequência em inicializar tudo como zero? Dica: a inicialização pode usar o valor de γ.

   (b) Compare com os métodos baseados em modelo: qualidade da política aprendida ao longo do treino, número de passos de interação e tempo de execução.

6 Entrega

Código e um relatório em PDF com os gráficos e as respostas às perguntas. Por favor, use o grupo do WhatsApp para tirar dúvidas ou relatar problemas com o código fornecido. É a primeira vez que elaboramos esse exercício.

7 Uso de IA

A formulação do problema e a especificação das interfaces foram feitas sem IA. O código, porém, foi em sua maioria gerado automaticamente por uma LLM + harness. Como comentado anteriormente, se você quiser converter o código fornecido para outra linguagem da sua escolha, sinta-se livre para fazê-lo também com assistência de IA. Usar IA para tirar dúvidas do código também é permitido. No entanto, não use IA para nenhuma outra coisa, em particular, na implementação do que está listado na Seção 4 e na lógica dos experimentos.